

MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software and Automation Engineering

Report

Laboratory work No. 8

The Same Calls, Inside Your Own OS

Ciur Alexandrina, st. gr. FAF-241

Checked by:
Reițman Patricia, *University Assistant*
Department of Software and Automation Engineering,
FCIM Faculty, UTM

Chișinău – 2026

Contents

1 LABORATORY OBJECTIVES AND THEORY **1**
1.1 Objective 1
1.2 Theoretical Background 1

2 PRACTICAL WORK AND OBSERVATIONS **2**
2.1 Part 1: Process Creation in xv6 (run.c) 2
2.1.1 Observe Answers (Part 1) 2
2.2 Part 2: Inter-Process Communication (pingpong.c) 3
2.2.1 Observe Answers (Part 2) 3

3 CONCLUSIONS **4**

1 LABORATORY OBJECTIVES AND THEORY

1.1 Objective

The goal of this laboratory work is to investigate process control system calls (`fork`, `exec`, `wait`) directly within the xv6 teaching operating system, compare their behavior with standard Linux implementations, and implement bidirectional inter-process communication using dual pipe structures in `pingpong.c`.

1.2 Theoretical Background

The xv6 kernel implements minimal POSIX-compliant system calls:

- **Process Management (`fork`, `exec`, `wait`):** `fork()` creates a child process by duplicating kernel state. `exec()` loads an ELF executable from the file system, replacing memory contents. `wait()` suspends parent execution until a child terminates.
- **Unidirectional Pipes (`pipe`):** Pipes are kernel-managed buffers. Because each pipe supports single-direction data flow, bidirectional communication requires two independent pipe structures (`p2c` and `c2p`).
- **Blocking Read Operations:** Invoking `read()` on an empty pipe descriptor causes the calling process to sleep until the writer process inserts data via `write()`.

2 PRACTICAL WORK AND OBSERVATIONS

2.1 Part 1: Process Creation in xv6 (run.c)

The `user/run.c` utility demonstrates the three-step execution lifecycle of commands within the xv6 operating system.

```
$ run echo hello from the child
hello from the child
child exited with status 0
$ run nosuchprogram
exec nosuchprogram failed
child exited with status 1
```

Listing 2.1: Execution of run.c in xv6 with valid and non-existent programs

2.1.1 Observe Answers (Part 1)

1. **System Call Comparison:** xv6 utilizes `fork()`, `exec()`, and `wait()`, whereas Linux utilizes `fork()`, `execlp()`, and `waitpid()`. The core mechanics of process replication, image replacement, and parent synchronization remain identical.

2. **Output Origin:** The text `"hello from the child"` is printed by the `echo` utility after `exec()` overwrites the child memory space.

3. **Exit Status Discrepancy:** The exit status reported to the parent reflects the argument passed to `exit()`. In xv6, `user/run.c` explicitly calls `exit(1)` upon `exec` failure, whereas the Linux program called `exit(127)`.

2.2 Part 2: Inter-Process Communication (pingpong.c)

The `user/pingpong.c` program creates two processes that exchange a byte token using two unidirectional pipes.

```
$ pingpong
4: received ping
3: received pong
```

Listing 2.2: Execution of pingpong in xv6 demonstrating process synchronization

2.2.1 Observe Answers (Part 2)

1. **Dual Pipe Requirement:** UNIX pipes are unidirectional channels. Two pipes are required to prevent race conditions where a process reads back its own written byte before the recipient processes it.

2. **Synchronization Mechanism:** The parent is synchronized by the blocking `read(c2p[0], &b, 1)` call, which suspends parent execution until the child executes `write(c2p[1], &b, 1)`.

3 CONCLUSIONS

In this laboratory work, I verified that process management mechanisms in the minimal xv6 kernel closely mirror standard POSIX implementations in full Linux distributions. Implementing `run.c` reinforced how shell execution relies on `fork()`, `exec()`, and `wait()`.

Additionally, developing `pingpong.c` provided practical experience with kernel-level pipe buffers, descriptor cleanup, and process synchronization driven by blocking I/O calls.